Language Modeling, BERT, GPT & Friends: What If Someone Already Trained a Transformer for You?

KAUST–Mawhiba IOAI Training

Week 3 · Day 18

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

Table of Contents

Quick Recap: What We Built Yesterday

- **Day 17:** We built a Transformer from scratch in PyTorch
  - Self-attention: each word looks at every other word
  - Multi-head attention: multiple "perspectives" at once
  - Positional encoding: teaching the model word order
  - Encoder block: self-attention + feed-forward + residuals
- **Key insight:** Transformers can process all words *in parallel*
  - Much faster than RNNs (which go word by word)
  - Much better at capturing long-range dependencies
- **Today's question:** What if someone already trained a *massive* Transformer on billions of words — for you?

Today's Big Picture

- **Language Modeling:** Teaching a computer to understand and generate text
- We'll meet **four famous models**, all built on the Transformer:
  1. **BERT** — the fill-in-the-blank expert (encoder-only)
  2. **GPT** — the story writer (decoder-only)
  3. **T5** — the text-to-text converter (encoder-decoder)
  4. **BART** — the sentence fixer (encoder-decoder)
- Then we'll learn **how these models generate text**:
  - Greedy decoding, beam search, top-k sampling, temperature

Running Example

Throughout today, we'll follow a sentence about **Whiskers the cat**: "The cat sat on the _____". Every concept will be illustrated with this example!

The Three Types of Transformer Models

Encoder-Only

Reads the *whole* sentence

Good at *understanding*

Example: **BERT**

Decoder-Only

Reads left-to-right

Good at *generating*

Example: **GPT**

Encoder-Decoder

Reads then writes

Good at *converting*

Example: **T5, BART**

Think of it like reading a book:

Encoder = reading and understanding | Decoder = writing a response

What is a Language Model?

- A **language model** is a program that learns the *patterns of language*
- It answers: "Given some words, what word(s) come next?"
- **You already use language models every day:**
  - Phone keyboard autocomplete: "I'm on my _____" → *way*
  - Google search suggestions
  - ChatGPT, Gemini, Claude
- **Formally:** A language model assigns a *probability* to each possible next word

Whiskers Example

"The cat sat on the _____"

mat → 35%	**floor** → 20%
chair → 15%	**roof** → 10%
table → 8%	**banana** → 0.01%

Two Ways to Learn Language

Fill in the Blank

(Masked Language Modeling)

- Hide a word in the *middle*
- Use words on *both sides* to guess
- Looks both left *and* right

Example

"The `[MASK]` sat on the mat"

Answer: **cat** (using "sat on the mat" as clue)

Predict What's Next

(Causal Language Modeling)

- Read words left to right
- Predict the *next* word
- Only looks at words *before*

Example

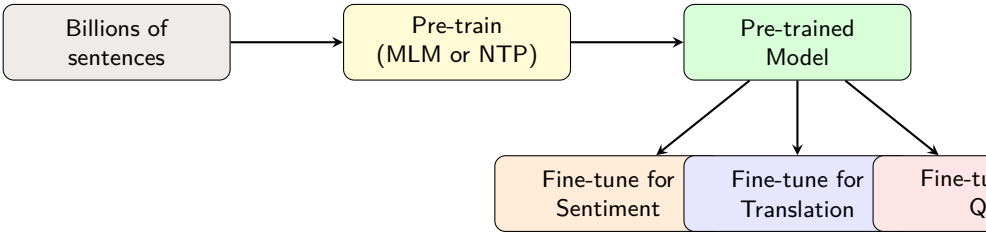
"The cat sat on the" $\rightarrow$???

Answer: **mat** (using only left context)

Why Does This Matter?

- **Pre-training:** Train on *billions* of sentences from the internet
  - The model learns grammar, facts, reasoning, common sense
  - No human labels needed — the text itself is the teacher!
- **Fine-tuning:** Take the pre-trained model, adapt it to *your* task
  - Sentiment analysis: "This movie is great!" $\rightarrow$ Positive
  - Translation: "Hello" $\rightarrow$ "Bonjour"
  - Question answering: "Where do cats sleep?" $\rightarrow$ "On mats"
- **The key idea:** *Learning language* $\Rightarrow$ *learning to think*
  - A model that can predict words well has learned a *lot* about the world

Pre-training + Fine-tuning Pipeline



- **Step 1:** Pre-train on massive text data (expensive, done once)
- **Step 2:** Fine-tune on your specific task (cheap, done many times)
- Same base model $\rightarrow$ many different applications!

Meet BERT

- **BERT** = *B*idirectional *E*ncoder *R*epresentations from *T*ransformers
- Created by Google in 2018 — a breakthrough moment in NLP
- **Key idea:** Use only the *encoder* part of the Transformer
  - Reads the *entire* sentence at once (bidirectional)
  - Every word can "see" every other word
- **Analogy:** BERT is like a student who reads the *whole* exam question before answering
  - GPT is like a student who covers the page and reads one word at a time
- **Trained on:**
  - English Wikipedia (2.5 billion words)
  - BookCorpus (800 million words)

Masked Language Modeling (MLM)

- **How BERT learns:** Hide some words, make the model guess them
- **Step 1:** Take a sentence and randomly replace 15% of words with `[MASK]`
- **Step 2:** Ask BERT to predict the masked words
- **Step 3:** Check if it got them right, update the model

Whiskers Example: MLM

Original: "The cat sat on the mat and licked its paws"

Masked: "The `[MASK]` sat on the `[MASK]` and licked its paws"

BERT predicts: $\texttt{[MASK]}_1 = \textbf{cat}$ ✓ $\texttt{[MASK]}_2 = \textbf{mat}$ ✓

BERT uses "sat on the" and "licked its paws" to figure out it's a cat on a mat!

Why Bidirectional Matters

- Consider: "The `[MASK]` went to the bank to deposit money"
- **Left context only** (GPT-style): "The _____ went to the"
  - Could be: man, woman, cat, dog, robot ...
- **Both sides** (BERT-style): "The _____ went to the bank to *deposit money*"
  - "deposit money" tells us it's a *person*: man, woman, customer ...
  - Also tells us "bank" means financial institution, not river bank!

Key Insight

Bidirectional = better understanding. When you read "The cat sat on the ___ and purred happily", the word "purred" *after* the blank helps you guess it's something comfortable (mat, sofa, lap).

Next Sentence Prediction (NSP)

- BERT has a *second* pre-training task: **Next Sentence Prediction**
- **Goal:** Does sentence B naturally follow sentence A?
- **How it works:**
  - 50% of the time: B *is* the real next sentence → label = `IsNext`
  - 50% of the time: B is a *random* sentence → label = `NotNext`

Whiskers Example: NSP

Positive pair (`IsNext`):
A: "The cat sat on the mat." B: "It purred softly." ✓

Negative pair (`NotNext`):
A: "The cat sat on the mat." B: "Python is a programming language." ✗

BERT learns that cats purring follows cats sitting, but programming doesn't!

BERT's Input Format

- BERT uses **special tokens** to organize its input:

```
[CLS] The cat sat on the mat [SEP] It purred softly [SEP]
```

- `[CLS]` — "Classification" token at the start
  - Its output represents the *whole* input's meaning
  - Used for classification tasks (positive/negative, IsNext/NotNext)
- `[SEP]` — "Separator" token between sentences
  - Tells BERT where sentence A ends and sentence B begins
- `[MASK]` — Replaces hidden words during MLM training
- **BERT also adds:**
  - **Token embeddings:** meaning of each word
  - **Segment embeddings:** which sentence (A or B)
  - **Position embeddings:** where in the sequence

BERT Model Sizes

Property	BERT-Base	BERT-Large
Transformer layers	12	24
Hidden size	768	1024
Attention heads	12	16
Total parameters	**110M**	**340M**

- **110 million** parameters was considered *huge* in 2018!
  - For comparison: GPT-4 has ~1.8 *trillion* parameters
- BERT-Base is the standard for most tasks (good balance of speed and quality)
- **Fun fact:** 340M parameters $\approx$ trying to remember 340 million numbers
  - That's like memorizing every phone number in the USA!

Fine-tuning BERT for Your Task

- After pre-training, BERT understands language really well
- To use it for a *specific* task, just add a small layer on top:

Task	**How to use BERT**
Sentiment Analysis	Take `[CLS]` output → linear layer → positive/negative
Question Answering	Mark answer start & end positions in the text
Named Entity Recognition	Classify each token (person, location, etc.)
Text Similarity	Compare `[CLS]` vectors of two sentences

Whiskers Example: Sentiment

Input: `[CLS]` "The cat is absolutely adorable!" `[SEP]`

BERT's `[CLS]` output → Linear layer → **Positive** (98%)

BERT in PyTorch: Loading a Pre-trained Model

```python
from transformers import BertTokenizer, BertModel

# Load pre-trained BERT (already trained on Wikipedia!)
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")

# Tokenize our Whiskers sentence
text = "The cat sat on the mat"
inputs = tokenizer(text, return_tensors="pt")

print(inputs["input_ids"])
# tensor([[101, 1996, 4937, 2938, 2006, 1996, 13523, 102]])
#          [CLS]  the   cat   sat   on   the   mat  [SEP]
```

- `from_pretrained` downloads a model someone already trained
- The tokenizer converts words to numbers BERT understands
- 101 = `[CLS]`, 102 = `[SEP]` (special token IDs)

BERT in PyTorch: Fill in the Blank (MLM)

```python
from transformers import pipeline

# Create a fill-mask pipeline (uses BERT by default)
unmasker = pipeline("fill-mask", model="bert-base-uncased")

# Ask BERT to fill in the blank!
result = unmasker("The cat sat on the [MASK].")

for r in result[:3]:
    print(f"{r['token_str']:>10} -> {r['score']:.1%}")

#         mat -> 25.3%
#       floor -> 18.7%
#         bed -> 12.1%
```

- HuggingFace `pipeline` makes it super easy — just 3 lines!
- BERT correctly predicts “mat” as the most likely word
- All predictions make sense: mat, floor, bed are things cats sit on

BERT in PyTorch: Sentiment Classification

```python
from transformers import pipeline

# Sentiment analysis pipeline (uses fine-tuned BERT)
classifier = pipeline("sentiment-analysis")

# Classify our Whiskers sentences
texts = [
    "The cat is adorable and very fluffy!",
    "The cat scratched the sofa and broke a vase.",
    "The cat sat on the mat."
]

for text in texts:
    result = classifier(text)[0]
    print(f"{result['label']:>10} ({result['score']:.0%}) | {text}")

#    POSITIVE (99%) | The cat is adorable and very fluffy!
#    NEGATIVE (89%) | The cat scratched the sofa and broke a vase.
#    POSITIVE (69%) | The cat sat on the mat.
```

- This uses a BERT model that was *fine-tuned* on sentiment data
- Same base model, different task — that's the power of pre-training!

Meet GPT

- **GPT** = *G*enerative *P*re-trained *T*ransformer
- Created by OpenAI — GPT-1 in 2018, GPT-2 in 2019, GPT-3 in 2020, GPT-4 in 2023
- **Key idea:** Use only the *decoder* part of the Transformer
  - Reads text *left to right* only (unidirectional)
  - Each word can only "see" words that came *before* it
  - This is called **causal masking** (can't peek at the future!)
- **Analogy:** GPT is like a storyteller writing one word at a time
  - It decides the next word based only on what it has written so far
  - BERT reads the whole page; GPT writes the page

Next Token Prediction (NTP)

- **How GPT learns:** Read words left to right, predict the *next* word
- At every position, GPT tries to predict what comes next:

Whiskers Example: Next Token Prediction

Input so far	GPT predicts next
"The"	→ "cat" ✓
"The cat"	→ "sat" ✓
"The cat sat"	→ "on" ✓
"The cat sat on"	→ "the" ✓
"The cat sat on the"	→ "mat" ✓
"The cat sat on the mat"	→ "." ✓

- Every word becomes *both* a training example and a label!
- From one sentence, GPT gets 6 training examples for free

Causal Masking: No Peeking!

- **Problem:** In self-attention, every word looks at every other word
- **But GPT can't look at future words!** (that would be cheating)
- **Solution:** Use a *causal mask* — block attention to future tokens

Attention mask for "The cat sat on":

	The	cat	sat	on
The	✓	✗	✗	✗
cat	✓	✓	✗	✗
sat	✓	✓	✓	✗
on	✓	✓	✓	✓

- ✓= can see ✗= blocked (future)
- "sat" can see "The" and "cat" but NOT "on"
- This is the *only* difference between BERT and GPT attention!

How GPT Generates Text

- GPT generates text **one word at a time** (autoregressive):

Whiskers Example: Generation

1. Start with: "The cat"
2. GPT predicts: "sat" → "The cat sat"
3. GPT predicts: "on" → "The cat sat on"
4. GPT predicts: "the" → "The cat sat on the"
5. GPT predicts: "mat" → "The cat sat on the mat"
6. GPT predicts: "." → "The cat sat on the mat."
7. GPT predicts: "It" → "The cat sat on the mat. It"
8. GPT predicts: "purred" → "The cat sat on the mat. It purred"
9. ...keeps going until it predicts a stop token!

- Each new word is *fed back* as input for the next prediction

GPT Family: Getting Bigger and Smarter

Model	Year	Parameters	Training Data
GPT-1	2018	117M	BookCorpus
GPT-2	2019	1.5B	WebText (40GB)
GPT-3	2020	175B	570GB of text
GPT-4	2023	~1.8T	Unknown (huge)

- Each version: *bigger model* + *more data* = *better results*
- GPT-3 is **1000x** bigger than GPT-1!
- **Emergent abilities:** Bigger models can do things smaller ones can't
  - GPT-1: basic text completion GPT-3: few-shot learning, code
  - GPT-4: reasoning, image understanding, passing exams

GPT in PyTorch: Text Generation

```python
from transformers import pipeline

# Create a text generation pipeline with GPT-2
generator = pipeline("text-generation", model="gpt2")

# Give GPT-2 a prompt about Whiskers
result = generator(
    "The cat sat on the mat and",
    max_new_tokens=20,
    do_sample=False  # greedy (pick most likely word)
)

print(result[0]["generated_text"])
# "The cat sat on the mat and looked up at the ceiling.
#  It was a beautiful day, and the sun was shining."
```

- GPT-2 continues our Whiskers story naturally!
- `do_sample=False`: always pick the most probable next word
- We'll learn fancier generation strategies later (beam search, sampling)

GPT in PyTorch: Under the Hood


```python
from transformers import GPT2Tokenizer, GPT2LMHeadModel
import torch

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")

# Encode our prompt
input_ids = tokenizer.encode("The cat sat on the",
                             return_tensors="pt")

# Get predictions for the NEXT token
with torch.no_grad():
    outputs = model(input_ids)
    next_token_logits = outputs.logits[0, -1, :]  # last position

# Top 5 predictions
top5 = torch.topk(next_token_logits, 5)
for score, idx in zip(top5.values, top5.indices):
    word = tokenizer.decode(idx)
    print(f"  {word:>10} (score: {score:.2f})")
```

- `logits[0, -1, :]` = predictions at the *last* position = next word
- Higher score = more likely word

BERT vs GPT: Side by Side

Feature	BERT	GPT
Architecture	Encoder only	Decoder only
Direction	Bidirectional (sees all words)	Left-to-right only
Training task	Masked LM (fill blanks) + NSP	Next token prediction
Good at	Understanding text	Generating text
Use cases	Classification, QA, NER	Text completion, chat, code
Analogy	Reading comprehension	Creative writing

Whiskers Example

BERT: "The `[MASK]` sat on the mat" $\rightarrow$ predicts: **cat**
GPT: "The cat sat on the" $\rightarrow$ continues: **mat. It purred softly...**

BERT fills in gaps. GPT continues the story.

When to Use Which?

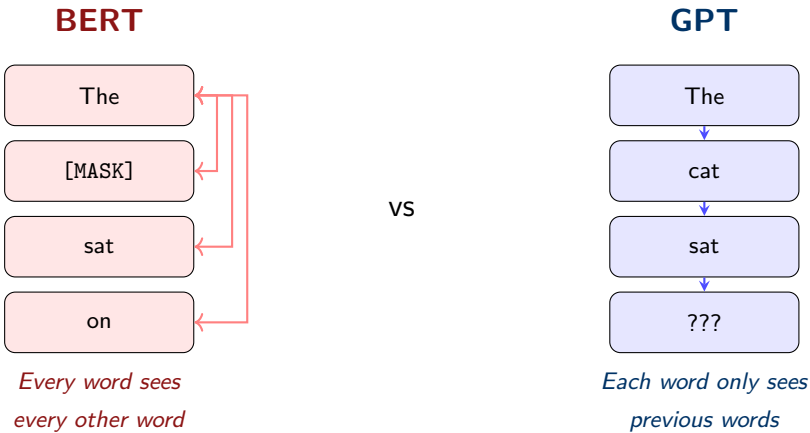
Use BERT when...

- You need to *classify* text
  - Spam detection
  - Sentiment analysis
- You need to *extract* information
  - Named entity recognition
  - Question answering
- You need to *compare* sentences
  - Semantic similarity
  - Paraphrase detection

Use GPT when...

- You need to *generate* text
  - Story writing
  - Code generation
- You need a *chatbot*
  - Customer service
  - Virtual assistant
- You need *few-shot* learning
  - Tasks with very few examples
  - Zero-shot classification

The Encoder vs Decoder Visual



Meet T5: Everything is Text-to-Text

- **T5** = *T*ext-*t*o-*T*ext *T*ransfer *T*ransformer
- Created by Google in 2019
- **Key idea:** Treat *every* NLP task as "text in, text out"
  - Classification? Text in $\rightarrow$ label text out
  - Translation? English text in $\rightarrow$ French text out
  - Summarization? Long text in $\rightarrow$ short text out
  - Question answering? Question + context in $\rightarrow$ answer text out
- **Architecture:** Full encoder-decoder Transformer
  - Encoder reads and understands the input
  - Decoder generates the output, one word at a time
- **Trick:** Add a *task prefix* to tell T5 what to do!

T5: One Model, Many Tasks

- T5 uses a simple prefix to switch between tasks:

Whiskers Example: T5 Task Prefixes

Translation:
Input: `translate English to French: The cat sat on the mat`
Output: `Le chat s'est assis sur le tapis`

Summarization:
Input: `summarize: The cat sat on the mat. It purred softly and fell asleep. The owner came home and found the cat sleeping peacefully.`
Output: `A cat fell asleep on a mat.`

Sentiment:
Input: `classify: The cat is absolutely adorable!`
Output: `positive`

T5 Pre-training: Span Corruption

- T5's pre-training task: **Span Corruption** (a fancier version of MLM)
- Instead of masking one word, mask a *span* of consecutive words
- Replace each span with a special token like `<extra_id_0>`

Whiskers Example: Span Corruption

Original: "The cat sat on the mat and purred softly"

Corrupted input: "The `<extra_id_0>` the mat `<extra_id_1>` softly"

Target output: "`<extra_id_0>` cat sat on `<extra_id_1>` and purred"

T5 learns to reconstruct the missing spans!

- This trains *both* the encoder (understanding) and decoder (generation)
- Better than single-word masking: learns about phrases and word groups

T5 in PyTorch: Translation

```python
from transformers import pipeline

# T5 for translation
translator = pipeline("translation_en_to_fr", model="t5-small")

result = translator("The cat sat on the mat.")
print(result[0]["translation_text"])
# "Le chat s'est assis sur le tapis."
```

```python
# T5 for summarization
summarizer = pipeline("summarization", model="t5-small")

text = """The cat sat on the mat all morning. It watched
the birds through the window. When the owner came home,
the cat jumped up and rubbed against their legs.
The owner gave the cat some treats."""

result = summarizer(text, max_length=30)
print(result[0]["summary_text"])
# "the cat sat on the mat all morning and watched
#  birds through the window."
```

T5 in PyTorch: The Text-to-Text Way

```python
from transformers import T5Tokenizer, T5ForConditionalGeneration

tokenizer = T5Tokenizer.from_pretrained("t5-small")
model = T5ForConditionalGeneration.from_pretrained("t5-small")

# The prefix tells T5 what task to do!
tasks = [
    "translate English to German: The cat sat on the mat",
    "summarize: The cat is very fluffy and cute. It loves "
    "to sleep on the mat and purr all day long.",
]

for task in tasks:
    inputs = tokenizer(task, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=50)
    result = tokenizer.decode(outputs[0], skip_special_tokens=True)
    print(f"Input:  {task[:50]}...")
    print(f"Output: {result}\n")
```

- **Key insight:** The same `model.generate()` works for *any* task
- The prefix ("translate", "summarize") steers the model

Meet BART: Denoising Autoencoder for Text

- **BART** = *B*idirectional and *A*uto-*R*egressive *T*ransformer
- Created by Facebook/Meta in 2019
- **Key idea:** Combine BERT's encoder with GPT's decoder
  - **Encoder** (like BERT): reads the corrupted input bidirectionally
  - **Decoder** (like GPT): generates the clean output left-to-right
- **Training:** Corrupt a sentence in various ways, then reconstruct it
  - Token masking (like BERT's MLM)
  - Token deletion (remove words entirely)
  - Sentence permutation (shuffle sentences)
  - Text infilling (replace a span with a single `[MASK]`)
- **Especially good at:** summarization and text generation

BART Pre-training: Corrupt then Reconstruct

- BART tries many types of corruption during training:

Whiskers Example: BART's Corruption Tricks

Original: "The cat sat on the mat and purred"

1. **Token Masking:** "The `[MASK]` sat on the `[MASK]` and purred"
2. **Token Deletion:** "The sat on mat and purred" (words removed!)
3. **Text Infilling:** "The `[MASK]` the mat and purred" (span $\rightarrow$ one mask)
4. **Sentence Permutation:** "And purred. The cat sat on the mat"

In every case, BART must output the original sentence!

BART in PyTorch: Summarization


```python
from transformers import pipeline

# BART is especially good at summarization
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

article = """
The cat named Whiskers sat on the mat every morning
for three years. The owner tried moving the mat to
different rooms, but Whiskers always found it.
Veterinarians say cats often develop strong
attachments to specific spots in the home. Whiskers
became famous on social media, with millions of
followers watching the daily mat-sitting routine.
"""

result = summarizer(article, max_length=40, min_length=10)
print(result[0]["summary_text"])
# "Whiskers the cat has sat on the same mat every morning
#  for three years, becoming a social media sensation."
```

- BART captures the key points and writes a fluent summary
- `bart-large-cnn`: fine-tuned on CNN/DailyMail news articles

BART in PyTorch: Fill in Missing Text

```python
from transformers import BartTokenizer, BartForConditionalGeneration

tokenizer = BartTokenizer.from_pretrained("facebook/bart-base")
model = BartForConditionalGeneration.from_pretrained(
    "facebook/bart-base"
)

# BART can fill in missing spans (text infilling)
text = "The cat <mask> on the mat and <mask> softly"
inputs = tokenizer(text, return_tensors="pt")
outputs = model.generate(**inputs, max_new_tokens=20)

result = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(result)
# "The cat sat on the mat and purred softly"
```

- Unlike BERT (which fills one word), BART can fill *multi-word spans*
- The encoder understands the context; the decoder generates the fix
- Think of BART as an “auto-correct on steroids”

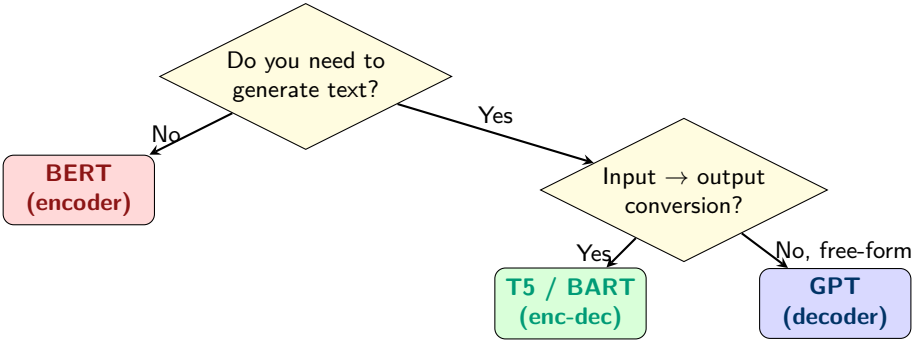
The Transformer Family Tree

	Encoder-Only	Decoder-Only	Encoder-Decoder
Models	BERT, RoBERTa	GPT, LLaMA	T5, BART
Direction	Bidirectional	Left-to-right	Both
Training	MLM + NSP	Next token	Span corruption
Best for	Understanding	Generating	Converting
Input	Full text	Prefix/prompt	Source text
Output	Labels/vectors	Continuation	Target text

Whiskers in All Three

BERT: "The `[MASK]` sat on the mat" → "cat"
GPT: "The cat sat on" → "the mat. It purred..."
T5: "translate: The cat sat on the mat" → "Le chat s'est assis sur le tapis"
BART: "The ___ on the mat and ___ softly" → "The cat sat on the mat

Which Architecture Should I Pick?



- **Need to understand/classify?** → Use BERT
- **Need to convert text?** (translate, summarize) → Use T5/BART
- **Need free-form generation?** (chat, stories) → Use GPT

The Generation Problem

- When GPT (or T5/BART decoder) generates text, at each step it produces **probabilities for every word** in its vocabulary
- A typical vocabulary has **50,000+** words/tokens
- **Question:** How do we pick which word to use?

Whiskers Example

After "The cat sat on the", GPT's probabilities might be:

mat: 25%	floor: 18%	bed: 12%	ground: 8%	table: 6%
sofa: 4%	chair: 3%	roof: 2%	moon: 0.01%	banana: 0.001%

50,000 words, each with a probability. Which one do we pick?

- Different strategies give *very different* results!
- Let's explore: **greedy**, **beam search**, **top-k**, **top-p**, and **temperature**

Strategy 1: Greedy Decoding

- **Simplest approach:** Always pick the word with the *highest probability*
- At each step: word $= \arg\max P(\text{word} \mid \text{previous words})$

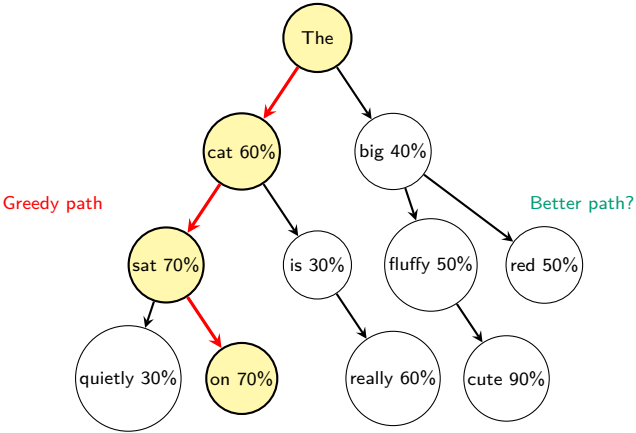
Whiskers Example: Greedy Decoding

Step	Context	Pick (highest prob)
1	"The cat sat on the"	**mat** (25%)
2	"The cat sat on the mat"	**.** (40%)
3	"The cat sat on the mat."	**The** (15%)
4	"The cat sat on the mat. The"	**cat** (20%)

Result: "The cat sat on the mat. The cat sat on the mat. The cat..."

- **Problem:** Greedy decoding often gets *stuck in loops!*
- Always picking the "safe" choice $\rightarrow$ boring, repetitive text
- Fast but not creative — like always ordering the same food

Greedy Decoding: Locally Optimal ≠ Globally Optimal



- **Greedy** always picks the highest probability at each step (red path)
- But "The big fluffy cute..." might have a *higher total probability*!
- Greedy misses good paths because it never looks ahead

Strategy 2: Beam Search

- **Idea:** Instead of keeping only the *best* word at each step, keep the *top k best sequences*
- k is called the **beam width** (usually 2–10)
- **How it works:**
  1. Start with the prompt
  2. Generate all possible next words
  3. Keep only the top k most probable *sequences* (not just words)
  4. Repeat for each of the k sequences
  5. At the end, pick the sequence with the highest *total* probability
- **Analogy:** Like exploring k paths in a maze simultaneously
  - Greedy = always turn right
  - Beam search = send k scouts down different paths, pick the best

Beam Search: Step by Step

Beam width = 2 (keep top 2 sequences at each step)

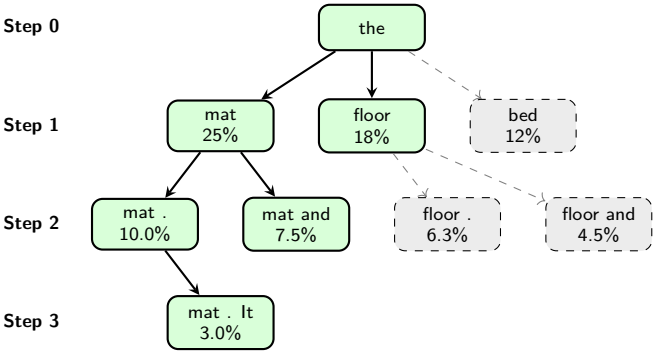
Whiskers Example: Beam Search (beam

Start: "The cat sat on the"

Step 1 — expand:
Beam 1: "...the **mat**" (prob: 0.25) ✓ kept
Beam 2: "...the **floor**" (prob: 0.18) ✓ kept
"...the bed" (prob: 0.12) ✗ dropped

Step 2 — expand both beams:
Beam 1a: "...mat **.**" ($0.25 \times 0.40 = 0.100$) ✓
Beam 1b: "...mat **and**" ($0.25 \times 0.30 = 0.075$)
Beam 2a: "...floor **.**" ($0.18 \times 0.35 = 0.063$)
Beam 2b: "...floor **and**" ($0.18 \times 0.25 = 0.045$)
Keep top 2: **"mat."** (0.100) and **"mat and"** (0.075)

Beam Search: Visual



- Green = kept (in the beam) Gray dashed = pruned
- We explore multiple paths but keep it manageable

Beam Width Trade-offs

Property	Small beam (2–3)	Large beam (10+)
Speed	✓ Fast	✗ Slow
Quality	Good enough	Better (diminishing returns)
Diversity	More diverse	Less diverse (converges)
Memory	Low	High

- **Beam = 1** is just greedy decoding!
- **Beam = 5** is the most common choice in practice
- **When to use beam search:**
  - ✓ Translation (need accuracy, not creativity)
  - ✓ Summarization (need to capture key facts)
  - ✗ Creative writing (too "safe" and boring)
  - ✗ Chatbots (responses feel robotic)

Beam Search in PyTorch

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

# Compare greedy vs beam search
prompt = "The cat sat on the mat and"

# Greedy (beam=1): fast but repetitive
greedy = generator(prompt, max_new_tokens=30,
                   num_beams=1, do_sample=False)
print("Greedy:", greedy[0]["generated_text"])

# Beam search (beam=5): better quality
beam = generator(prompt, max_new_tokens=30,
                 num_beams=5, do_sample=False,
                 no_repeat_ngram_size=2)
print("Beam-5:", beam[0]["generated_text"])
```

- `num_beams=5`: keep 5 candidate sequences
- `no_repeat_ngram_size=2`: prevent repeating 2-word phrases
- Beam search avoids the repetition loops of greedy decoding!

Strategy 3: Temperature Scaling

- **Idea:** Control how "confident" or "creative" the model is
- The model outputs raw scores (logits) for each word
- We convert them to probabilities using **softmax**:

$$P(w_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- z_i = raw score for word i, T = *temperature*
- **Temperature controls the "sharpness" of probabilities:**
  - $T < 1$ (low): makes confident words *more* confident → *focused*
  - $T = 1$ (default): original probabilities
  - $T > 1$ (high): makes all words more *equal* → *creative/random*

Temperature: Visual Intuition

Whiskers Example: Temperature Effect

After "The cat sat on the", original probabilities (T=1.0):

	mat	floor	bed	moon
T = 0.1 (frozen)	**99.5%**	0.4%	0.1%	0.0%
T = 0.5 (cool)	**62%**	25%	10%	0.01%
T = 1.0 (default)	**25%**	18%	12%	0.1%
T = 2.0 (warm)	**15%**	14%	12%	2%
T = 5.0 (hot)	8%	7%	7%	5%

- **Low T** $\rightarrow$ Almost always picks "mat" (safe, predictable)
- **High T** $\rightarrow$ Might pick "moon"! (wild, surprising, sometimes nonsense)
- **Analogy:** Temperature is like a creativity dial
  - Low = careful student who always gives the textbook answer
  - High = creative artist who sometimes says something wild

Temperature: When to Use What?

Temperature	Behavior	Good for
$T = 0$ (greedy)	Always most likely word	Factual QA, code
$T = 0.3$	Very focused	Translation, summaries
$T = 0.7$	Balanced	General chatbots
$T = 1.0$	Original distribution	Default / baseline
$T = 1.5+$	Very random	Brainstorming, poetry

Whiskers at Different Temperatures

T = 0.2: "The cat sat on the mat. The cat sat on the mat." (boring!)
T = 0.7: "The cat sat on the mat and purred happily." (nice!)
T = 1.5: "The cat sat on the mat, dreaming of distant galaxies." (creative!)
T = 3.0: "The cat sat on the purple bicycle singing opera." (nonsense!)

Temperature in PyTorch

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")
prompt = "The cat sat on the mat and"

# Try different temperatures
for temp in [0.3, 0.7, 1.0, 1.5]:
    result = generator(
        prompt,
        max_new_tokens=20,
        do_sample=True,      # enable sampling
        temperature=temp,    # creativity dial
    )
    text = result[0]["generated_text"]
    print(f"T={temp}: {text}")

# T=0.3: "The cat sat on the mat and looked up at the window."
# T=0.7: "The cat sat on the mat and began to purr softly."
# T=1.0: "The cat sat on the mat and stretched its tiny paws."
# T=1.5: "The cat sat on the mat and whispered ancient secrets."
```

- `do_sample=True`: enables random sampling (required for temperature)
- Higher temperature → more varied and surprising outputs

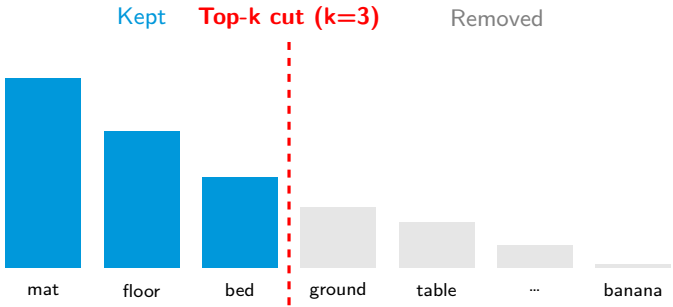
Strategy 4: Top-k Sampling

- **Problem with temperature:** The model *could* still pick very unlikely words
- **Top-k idea:** Only consider the *top k most likely words*, ignore the rest
- **How it works:**
  1. Get probabilities for all 50,000 words
  2. Keep only the top k (e.g., $k = 50$), set rest to 0
  3. Re-normalize so the top k sum to 1, then sample

Whiskers Example: Top-k

All words: mat(25%), floor(18%), bed(12%), ground(8%), ..., banana(0.001%)
Top-3 only: mat(**45%**), floor(**33%**), bed(**22%**) (re-normalized)
Banana can NEVER be picked!

Top-k: Visual



- **Benefit:** Prevents picking nonsense words
- **Common values:** $k = 40$ (GPT-2 default), $k = 50$
- **Limitation:** Fixed k doesn't adapt to how confident the model is

Strategy 5: Top-p (Nucleus) Sampling

- **Problem with top-k:** Sometimes only 2 good words, sometimes 10. Fixed k doesn't adapt!
- **Top-p idea:** Keep the *smallest set of words* whose probabilities sum to $\geq p$
- **Steps:** Sort by probability $\rightarrow$ add words until cumulative $\geq p \rightarrow$ sample

Whiskers Example: Top-p

mat(25%→25%) ✓ | floor(18%→43%) ✓ | bed(12%→55%) ✓ | ground(8%→63%) ✓ | table(6%→69%) ✓ | ...until 90%

Top-p auto-adjusts: more words when uncertain, fewer when confident!

Top-k vs Top-p

Feature	Top-k	Top-p (nucleus)
What it fixes	Number of words (k)	Probability mass (p)
Adaptive?	No (always k words)	Yes (varies per step)
Confident model	May include bad words	Keeps only 2–3 words ✓
Uncertain model	May miss good words	Keeps many words ✓
Typical value	$k = 50$	$p = 0.9$ or 0.95

- **In practice**, combine them: $T = 0.7$–0.9, top-$p = 0.9$–0.95, top-$k = 50$
- **ChatGPT/Claude** use a combination of these strategies!

Top-k and Top-p in PyTorch

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")
prompt = "The cat sat on the mat and"

# Top-k sampling (k=50)
result_k = generator(prompt, max_new_tokens=20,
                     do_sample=True, top_k=50)

# Top-p (nucleus) sampling (p=0.92)
result_p = generator(prompt, max_new_tokens=20,
                     do_sample=True, top_p=0.92)

# Combined: temperature + top-k + top-p (best practice)
result_all = generator(prompt, max_new_tokens=20,
                       do_sample=True,
                       temperature=0.8,
                       top_k=50,
                       top_p=0.95)

print(result_all[0]["generated_text"])
# "The cat sat on the mat and stretched lazily in the sun."
```

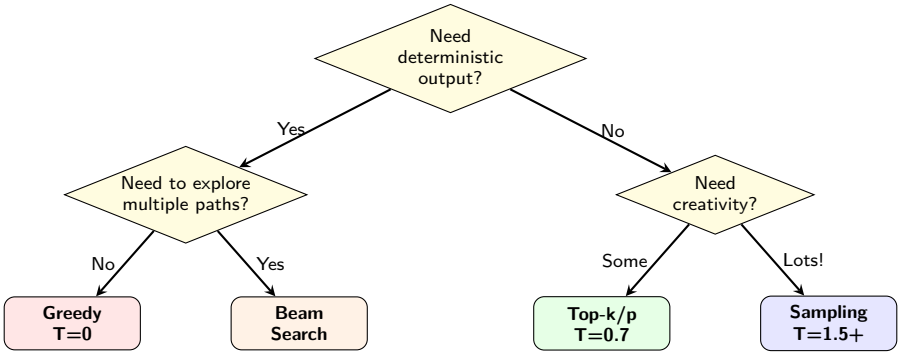
- Combining all three gives the best results in practice

Comparing All Strategies Side by Side

```python
from transformers import pipeline, set_seed

generator = pipeline("text-generation", model="gpt2")
prompt = "The cat sat on the mat and"
set_seed(42)  # for reproducibility

strategies = {
    "Greedy":      dict(do_sample=False),
    "Beam (5)":    dict(do_sample=False, num_beams=5,
                        no_repeat_ngram_size=2),
    "Top-k=50":    dict(do_sample=True, top_k=50),
    "Top-p=0.92":  dict(do_sample=True, top_p=0.92),
    "T=0.3":       dict(do_sample=True, temperature=0.3),
    "T=1.5":       dict(do_sample=True, temperature=1.5),
}

for name, params in strategies.items():
    result = generator(prompt, max_new_tokens=15, **params)
    print(f"{name:>12}: {result[0]['generated_text']}")
```

- Each strategy produces a different style of text!
- Try running this yourself in the lab

Generation Strategies: Decision Guide

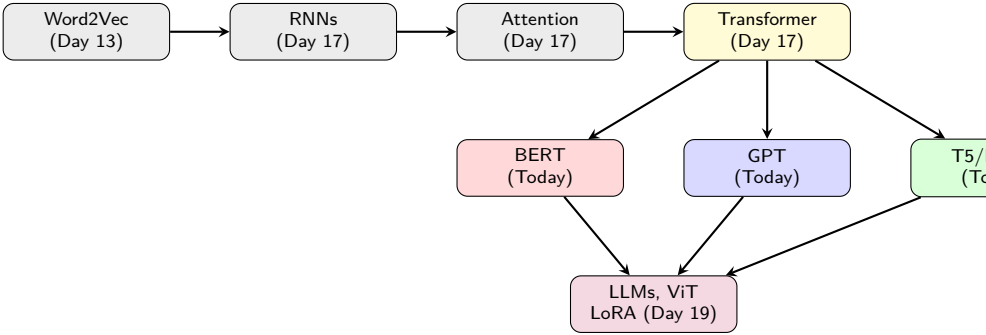


- **Code / factual QA:** Greedy or low temperature
- **Translation / summaries:** Beam search
- **Chatbots:** Top-k/p + moderate temperature
- **Creative writing:** High temperature + top-p

Today's Key Takeaways

1. **Language Modeling** = teaching a computer to predict words
   - Two approaches: fill-in-the-blank (MLM) and predict-next (NTP)
2. **BERT** (encoder-only): reads bidirectionally, great for *understanding*
   - Trained with MLM + NSP
3. **GPT** (decoder-only): reads left-to-right, great for *generating*
   - Trained with next token prediction
4. **T5** (encoder-decoder): everything is text-to-text
5. **BART** (encoder-decoder): corrupt then reconstruct
6. **Generation strategies:**
   - Greedy (fast, repetitive) $\rightarrow$ Beam search (better, still deterministic)
   - Temperature (creativity dial) $\rightarrow$ Top-k/Top-p (control randomness)

The Big Picture: What We've Learned



- We went from simple word vectors → sequential models → attention → Transformers → *pre-trained language models*
- **Tomorrow (Day 19):** LLMs, Vision Transformers, and LoRA

HuggingFace Cheat Sheet

Task	Pipeline	Model
Fill in blank	`fill-mask`	`bert-base-uncased`
Sentiment	`sentiment-analysis`	fine-tuned BERT
Text generation	`text-generation`	`gpt2`
Translation	`translation_en_to_fr`	`t5-small`
Summarization	`summarization`	`facebook/bart-large-cnn`
QA	`question-answering`	`deepset/roberta-base-squad2`

- **All you need:**

```
from transformers import pipeline
pipe = pipeline("task", model="model-name")
result = pipe("your input text")
```

- 3 lines of code to use models trained on billions of words!

Credits

KAUST Academy

King Abdullah University of Science and Technology

Language Modeling & Pre-trained Transformers content authored for the KAUST–Mawhiba IOAI 2026 Training Program

BERT (Devlin et al., 2018) · GPT (Radford et al., 2018)

T5 (Raffel et al., 2019) · BART (Lewis et al., 2019)

Week 3 · Day 18